🗺️ EventBook — Mapa Visual de Montagem

Documento de engenharia para alinhamento entre Backlog v3, Código Atual e Ordem de Construção.

Projeto: EventBook | Status Atual: M01-M03 (~90%) | Próxima Etapa: M04 (Lotes)

Documento de referência para a equipe de desenvolvimento. Mostra o estado atual do projeto, a ordem de construção de cada peça, as dependências entre issues e o caminho crítico até o release.

📍 Diagnóstico: Onde estamos agora

MILESTONE	PROGRESSO	STATUS
M01 Fundação e Infraestrutura	95%	Repo, Docker (3 serviços), Prisma, CI (4 jobs), lint, Dockerfiles — tudo pronto. Falta: Dockerfile do backend sem multi-stage, nginx sem config SPA, docker-compose unificado sem migration automática.
M02 Autenticação e RBAC	92%	Register, Login, JWT, middlewares auth/authorize, AuthContext, PrivateRoute, formulários — todos implementados. **8 testes de auth existem** (register, login, RBAC). Faltam testes de borda (e-mail duplicado 409, token expirado).
M03 Gestão de Eventos	88%	Schema `events`, CRUD completo na API (com RN-01), vitrine pública c/ paginação e busca, detalhe com badge de status, painel organizador com tabela+modal de criação/edição. **9 testes de eventos existem**. Faltam testes de borda e paginação.
M04 Lotes e Capacidade	10%	Schema `ticket_lots` **não existe** no Prisma. Nenhuma rota de lotes. UI tem placeholder com borda tracejada dizendo "Milestone 04".
M05 Pedido e Reserva	0%	Não iniciado.
M06 Checkout e Cancelamento	0%	Não iniciado.
M07 Staff e Check-in	5%	Rota `/checkin` existe no frontend como placeholder. Nenhum schema, API ou lógica.
M08 Ocupação	0%	Não iniciado.
M09 Hardening e Release	0%	Não iniciado.

> 📌 **IMPORTANT**
> O projeto está na **transição M03 → M04**. A fundação, autenticação e CRUD de eventos estão sólidos. A próxima fronteira é **Lotes e Capacidade** — tudo que vem depois (pedidos, checkout, check-in) depende dela.

🔀 Fluxo Macro: Caminho Crítico entre Milestones

O caminho que **não pode ser paralelizado** e define o prazo total do projeto:

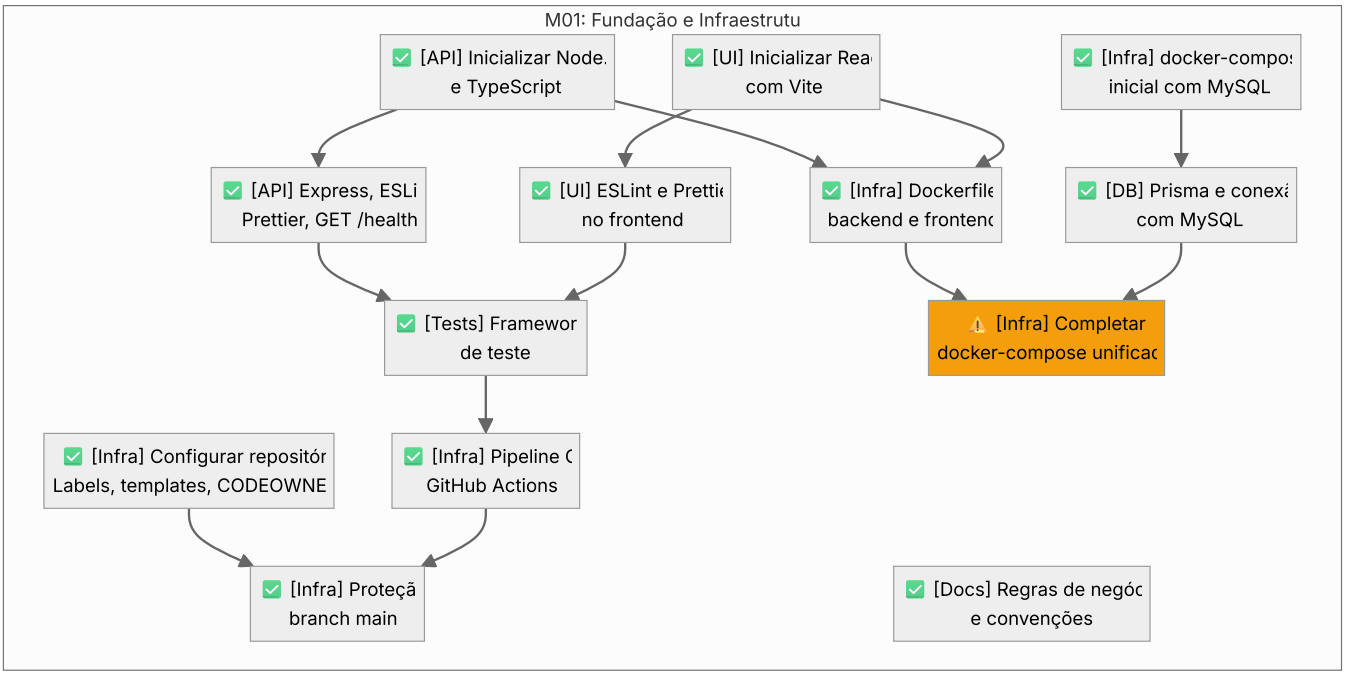


> **TIP**
> **Paralelismo possível:** M08 (Ocupação) pode ser desenvolvido em paralelo com M07 (Staff & Check-in), pois são independentes entre si. Ambos convergem para M09.

🧩 Mapa de Montagem Detalhado por Milestone

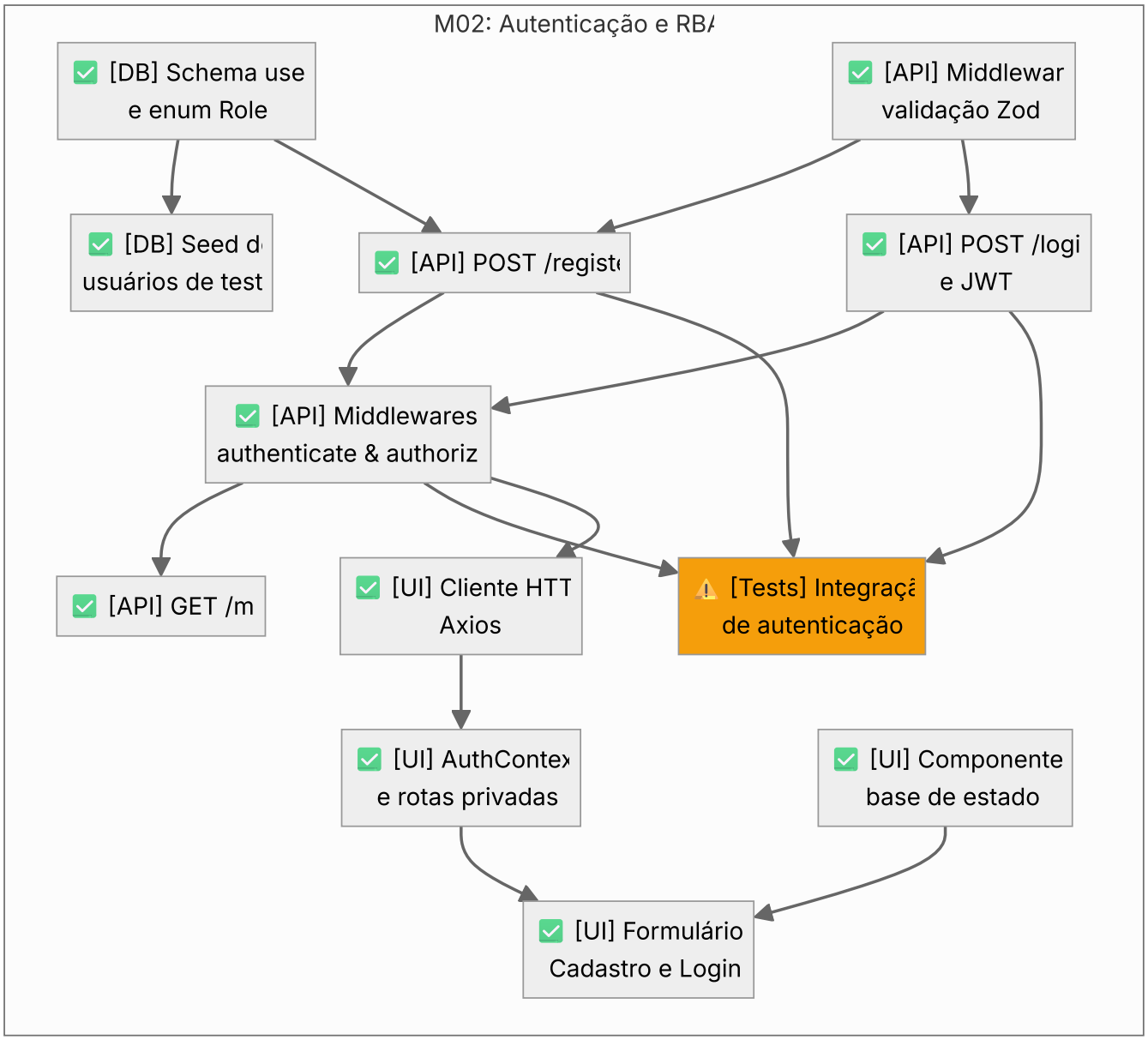
Cada diagrama abaixo mostra **as issues do milestone**, suas **dependências internas** (setas) e quais **issues podem rodar em paralelo** (sem setas entre si no mesmo nível).

M01 — Fundação e Infraestrutura ✅

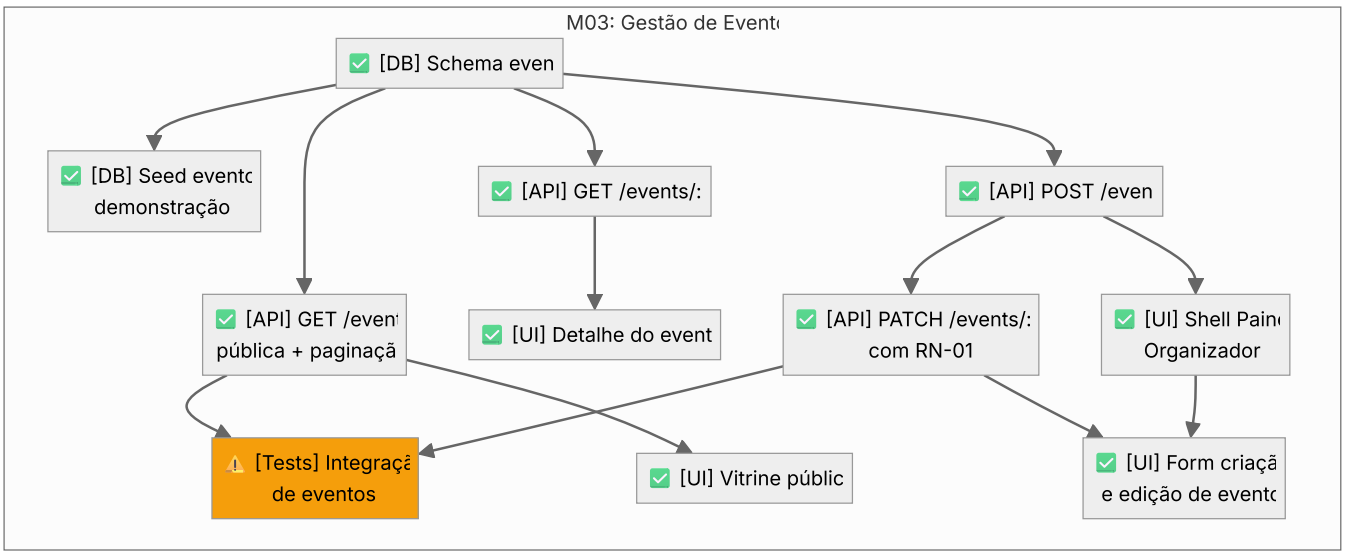


> **NOTE**
> **Issue pendente:** `[Infra] Completar docker-compose unificado` — validar que `docker-compose up` sobe frontend + backend + MySQL com migrations automáticas em um único comando.

M02 — Autenticação e RBAC ✅

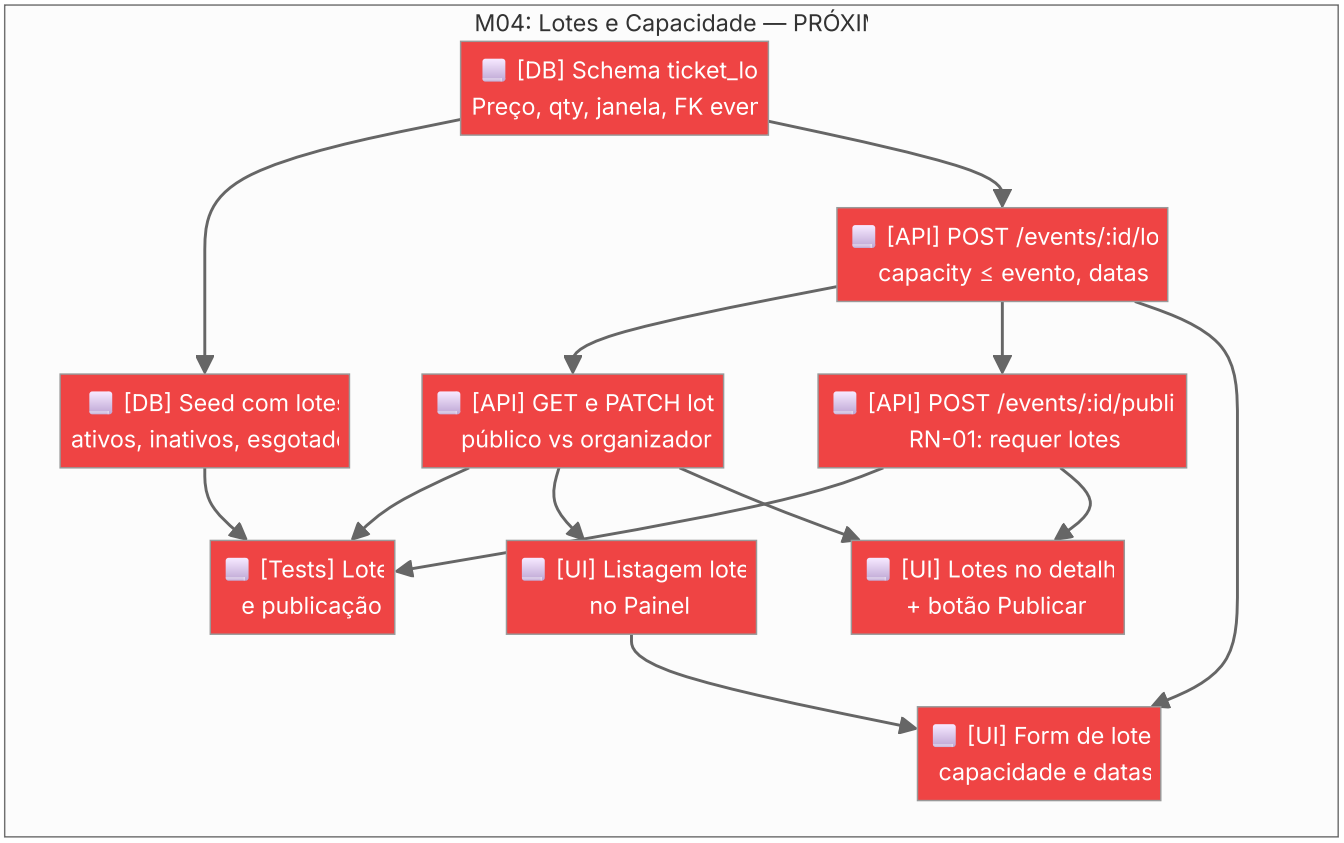


M03 — Gestão de Eventos ✅



M04 — Lotes e Capacidade 🔨 PRÓXIMO

> 📌 **IMPORTANT**
> Este é o **próximo milestone a atacar**. Nenhuma issue dele foi iniciada no código.

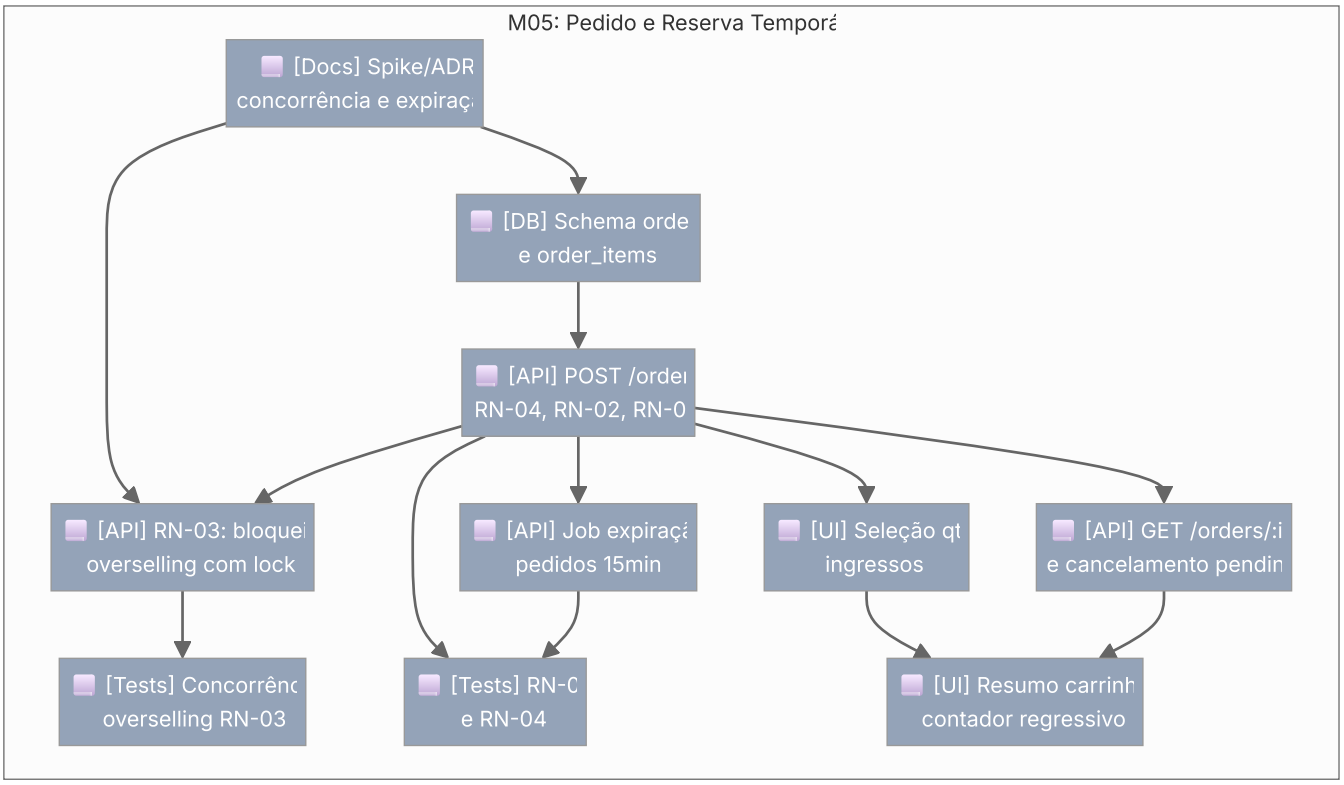


Ordem sugerida de execução:

1. `[DB] Schema ticket_lots` — migration é pré-requisito de tudo
2. `[DB] Seed com lotes` + `[API] POST /events/:id/lots` — podem rodar em paralelo

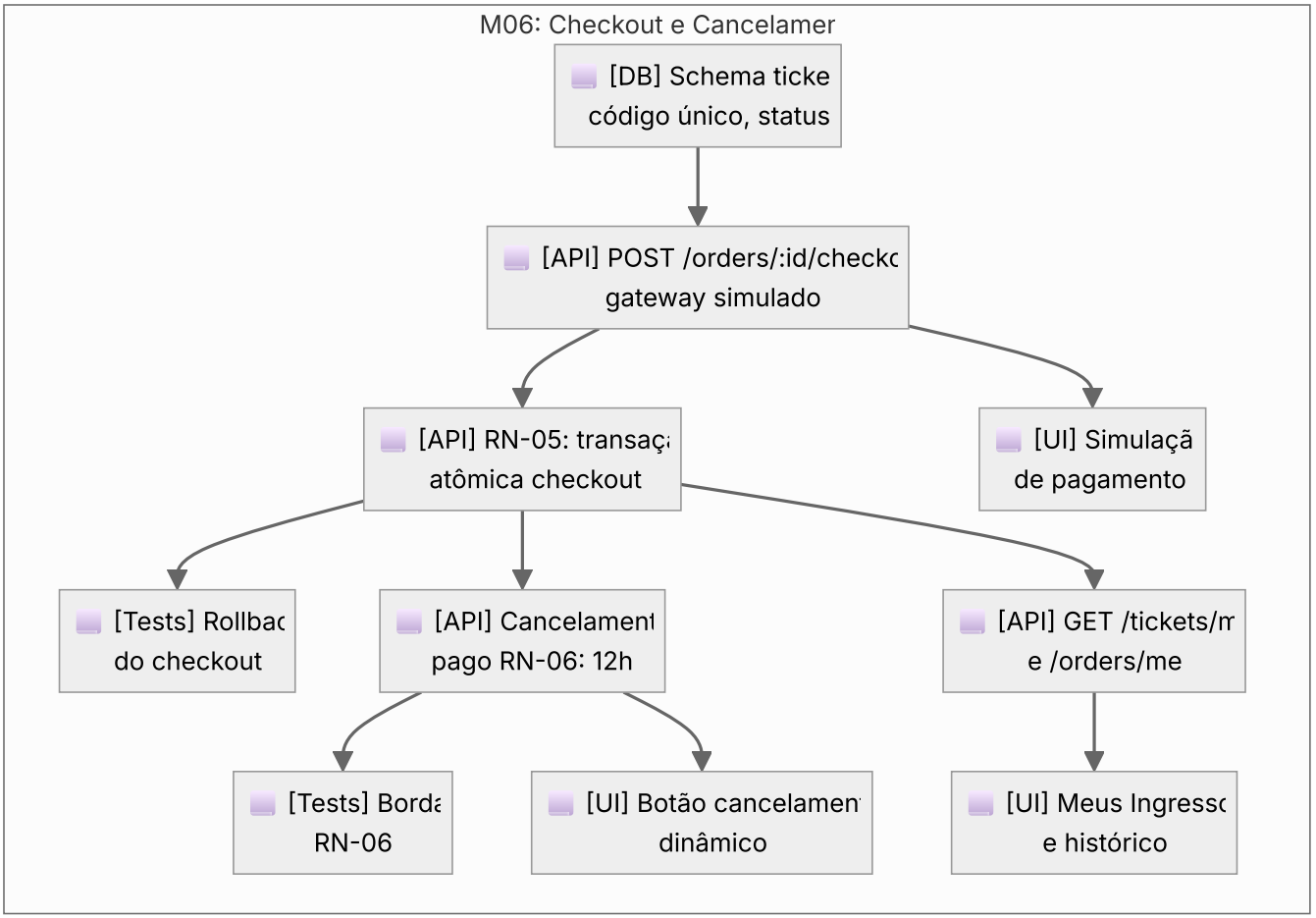
3. `[API] GET e PATCH lotes` + `[API] POST /events/:id/publish` — dependem do POST
4. `[UI] Listagem lotes` + `[UI] Form de lotes` — UI trabalha contra mocks enquanto API é construída
5. `[UI] Lotes no detalhe + botão Publicar`
6. `[Tests] Lotes e publicação` — ao final, testes cobrem tudo

M05 — Pedido e Reserva Temporária

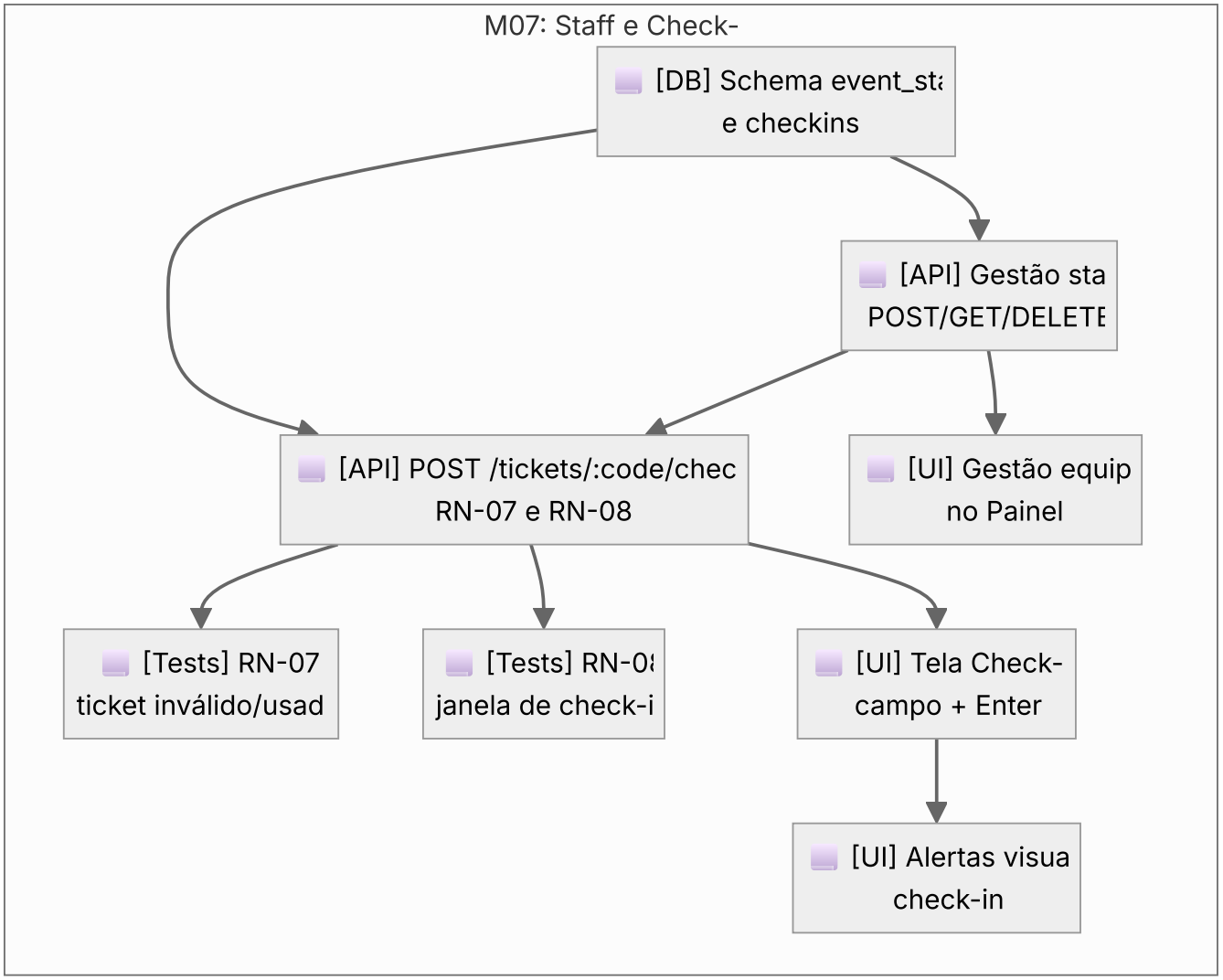


> ⚠️ **WARNING**
> **Spike obrigatório:** Antes de codificar qualquer coisa do M05, a decisão de lock pessimista vs update condicional vs contador **deve estar registrada em ADR**. Isso impacta o design do schema, da transação e dos testes de concorrência.

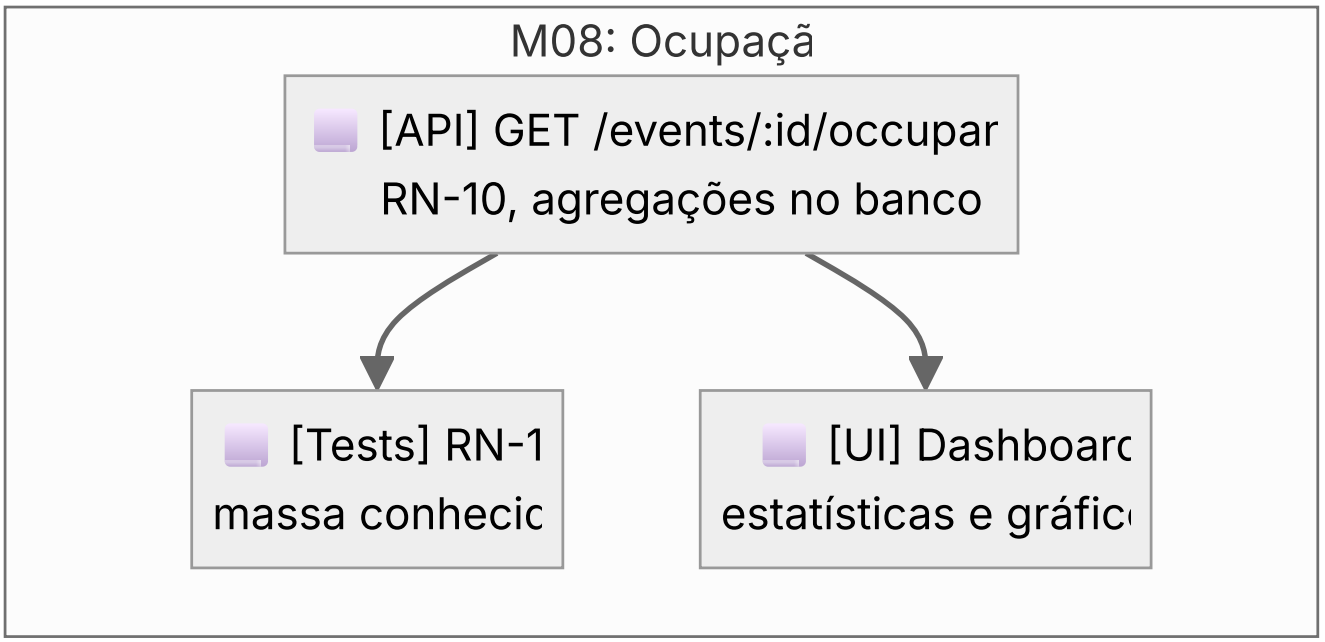
M06 — Checkout Atômico e Cancelamento



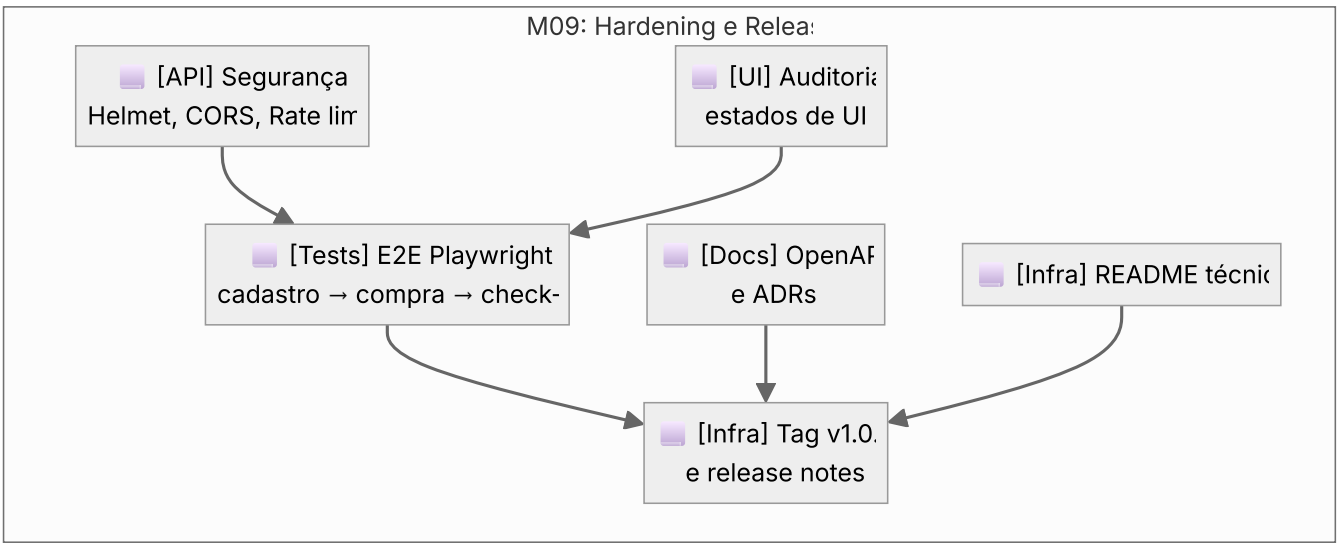
M07 — Staff e Check-in



M08 — Ocupação (pode rodar em paralelo com M07)

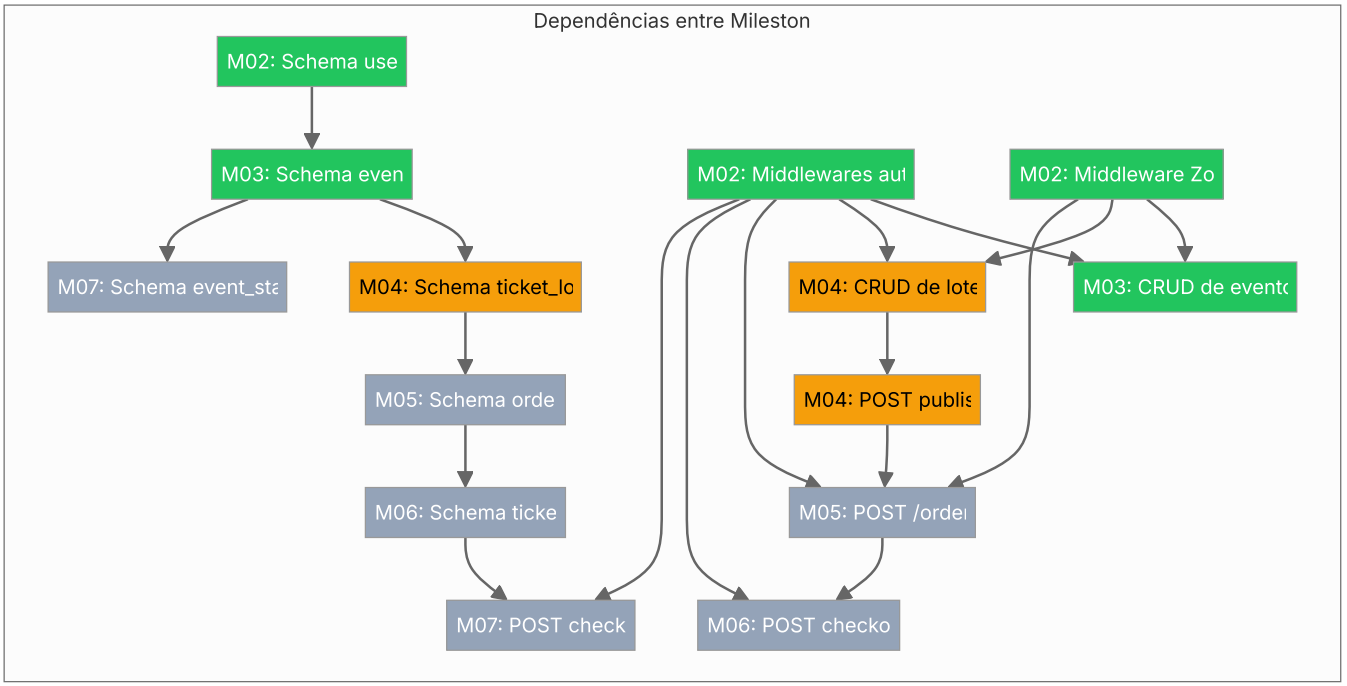


M09 — Hardening e Release



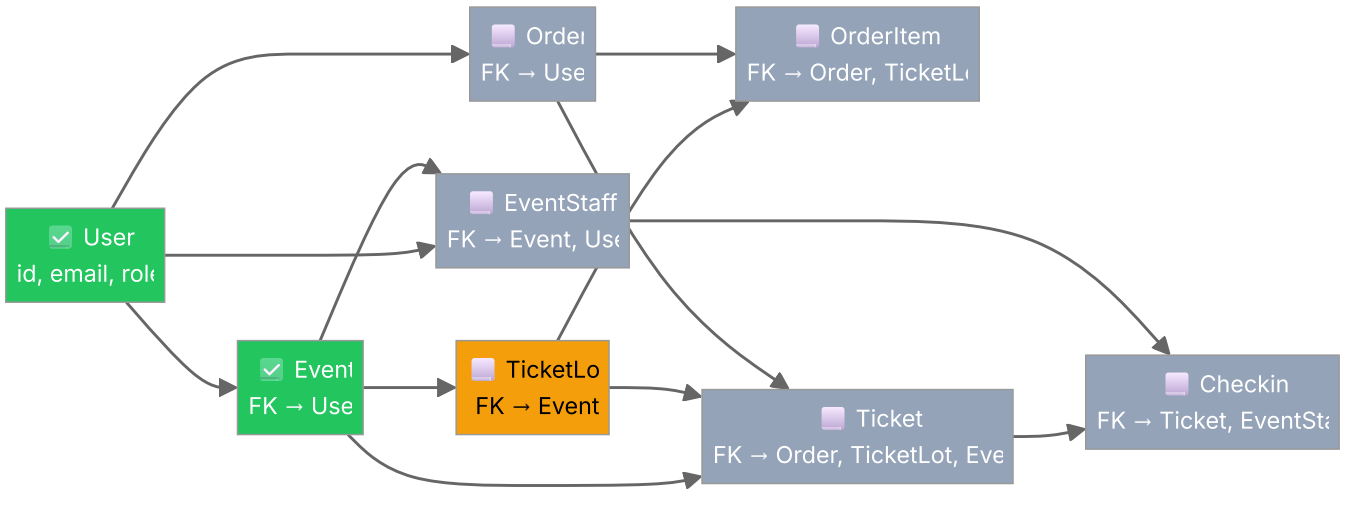
🔗 Mapa de Dependências Cross-Milestone

Este diagrama mostra como as issues **entre milestones diferentes** se conectam — a "cola" que une tudo:



🧱 Mapa de Dependências do Schema (Prisma)

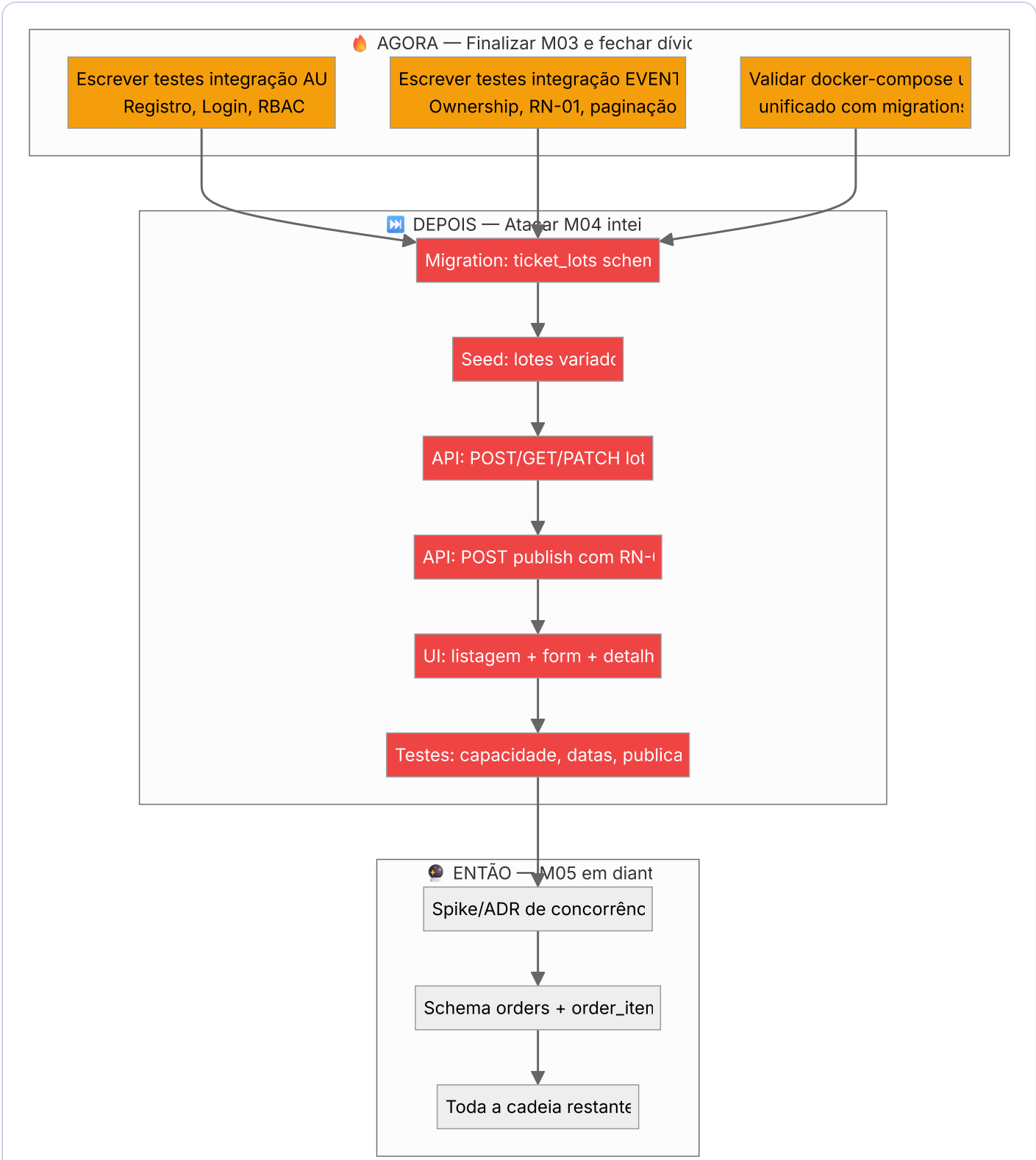
A ordem em que os modelos **devem** ser criados, pois cada um depende do anterior via FK:



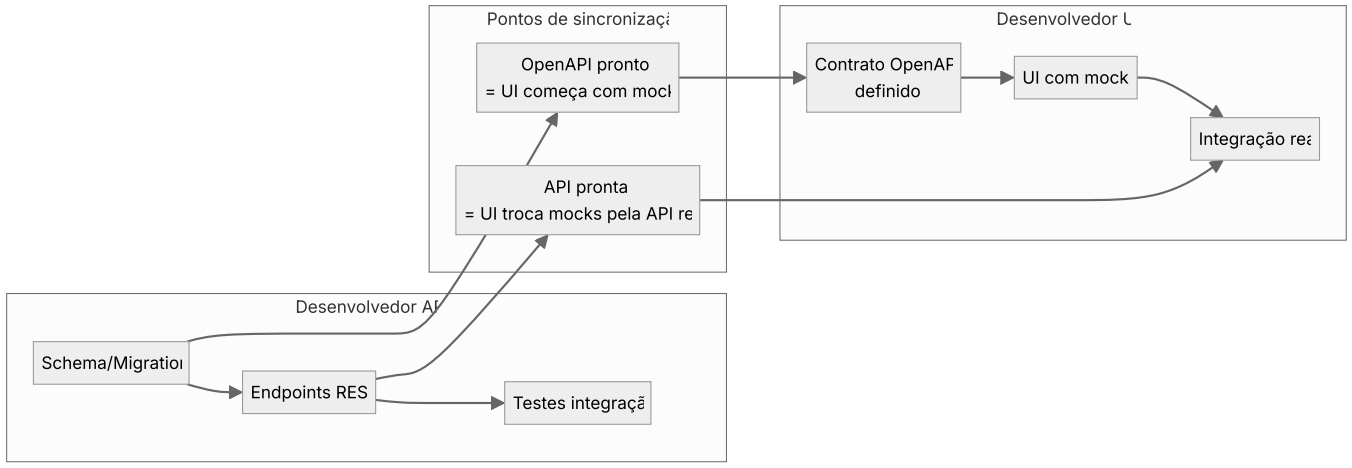
🚦 Regras de Negócio: Onde cada RN é implementada

RN	DESCRIÇÃO	MILESTONE	ISSUE DE IMPLEMENTAÇÃO	ISSUE DE TESTE
RN-01	Bloquear edição de evento no passado	M03	`PATCH /events/:id`	Testes M03
RN-01	Bloquear publicação sem lotes	M04	`POST /events/:id/publish`	Testes M04
RN-02	Validar janela de venda e estoque	M05	`POST /orders`	Testes RN-02/04
RN-03	Bloqueio de overselling com lock	M05	Lock transacional	Testes concorrência
RN-04	Expiração de reserva em 15min	M05	Job de expiração	Testes RN-02/04
RN-05	Transação atômica no checkout	M06	`POST /orders/:id/checkout`	Testes rollback
RN-06	Cancelamento 12h antes do evento	M06	Cancelamento pago	Testes bordas
RN-07	Validação de ticket no check-in	M07	`POST /tickets/:code/checkin`	Testes RN-07
RN-08	Janela de check-in (starts_at - 2h)	M07	`POST /tickets/:code/checkin`	Testes RN-08
RN-09	Snapshot de preço no pedido	M05	`POST /orders` (unit_price_snapshot)	Testes RN-02/04
RN-10	Dashboard de ocupação	M08	`GET /events/:id/occupancy`	Testes RN-10

📋 Plano de Ação Imediato: O que fazer agora



🏗️ Estratégia de Paralelismo (API + UI)



> 💡 **TIP**
> **Em cada milestone**, o contrato OpenAPI deve ser definido **no início**. Isso permite que a UI trabalhe contra mocks enquanto a API é construída, maximizando o paralelismo entre os desenvolvedores.

⚠️ Dívidas Técnicas Identificadas

Itens que não são do backlog, mas que a análise do código revelou e que devem ser resolvidos oportunisticamente:

#	ÁREA	PROBLEMA	QUANDO RESOLVER
1	**Backend Dockerfile**	Não é multi-stage — imagem final contém devDependencies e source. Usa `npm install` ao invés de `npm ci`. Roda como root.	M09 (Hardening)
2	**Frontend Dockerfile**	Nginx sem `try_files $uri /index.html` — rotas SPA quebram em refresh. Sem gzip/brotli.	M09 (Hardening)
3	**Docker Compose**	Senha root MySQL hardcoded. Sem network isolation. Sem migration automática no boot.	M01 (issue pendente)
4	**CI**	Sem coverage report. Job de test não sobe MySQL (testes atuais mockam Prisma). Sem E2E.	M05 (quando testes precisarem de DB real para concorrência)
5	**Frontend Layout**	Sem Navbar/Layout compartilhado — cada página reimplementa header.	M04 (ao criar mais páginas)
6	**Dashboard**	Carrega todos os eventos (pageSize: 50), não filtra por organizador. Pode mostrar eventos de outros.	M04 (ao adicionar lotes ao painel)
7	**Testes Backend**	Usam mocks do Prisma (não testam contra banco real).	M05 (quando RN-03 exige teste com DB real)
8	**Sem toast/notificação**	Feedback de sucesso/erro em formulários é inline — sem sistema de toast global.	M04 (ao criar form de lotes)
9	**HomePage**	Componente existe mas não está roteado (`/*` redireciona para `/events`).	Decisão da equipe

Legenda

COR	SIGNIFICADO
Verde	Implementado e funcional
Amarelo	Parcial ou pendência menor
Vermelho	Próximo a ser feito (prioridade)
Cinza	Futuro (aguarda dependências)